# SPANS-99: Best Practice Summary

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/12/2026

Definitive best practice settings for distributed tracing and span analysis. Each entry specifies the exact configuration.

---

## Table of Contents


1. DQL Syntax Rules
2. Filtering & Performance
3. Time Range & Cost
4. Duration & Aggregation
5. Trace Analysis
6. Service Dependencies
7. Security
8. OpenPipeline Span Configuration
9. Bucket Strategy & Sampling


---

## 1. DQL Syntax Rules

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Equality operator | `==` (double equals) — never `=` | Critical |
| Array literals | `{"a", "b"}` with curly braces and double quotes — never `('a', 'b')` | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| NULL checks | `isNull(field)` / `isNotNull(field)` — `field == null` is rejected (*"`null` isn't allowed here"*) | Critical |
| Multi-value matching | `in(field, {"val1", "val2"})` — never SQL `IN` | Critical |
| `span.kind` values | Lowercase only: `"server"`, `"client"`, `"internal"`, `"producer"`, `"consumer"` | Critical |
| `span.status_code` values | Lowercase `"error"` / `"ok"`; the field is **absent** when unset (most spans), so `!= "error"` does not count successes — use total minus errors | Critical |
| String literals | Double quotes everywhere — never single quotes | Critical |
| Grouping syntax | `summarize n = count(), by:{field}` with curly braces — never `GROUP BY` | Critical |
| Service field | `dt.service.name` (every span) — not `service.name` (OpenTelemetry spans only; 6.7% on the validation tenant). Joins and identity: `dt.smartscape.service` (`dt.entity.service` is deprecated) — names can repeat across applications | Critical |
| ID fields are `uid` | `trace.id == toUid("…")` — a plain string comparison is always false and returns zero rows | Critical |
| Always alias aggregation results | `summarize error_count = count()` — required for `sort` and `fieldsAdd` | Critical |

## 2. Filtering & Performance

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Filter immediately after fetch | All `filter` commands before any `fieldsAdd`, `summarize`, `sort` | Critical |
| Selective filters first | Grail has **no field indexes**; a filter that rules out most data lets it skip reading (1 h: no filter 910 MB, `dt.service.name == "checkout"` 54 MB, `trace.id == toUid(…)` 57 MB) | Critical |

| Practice | Recommended Setting/Value | Priority |
| --- | --- | --- |
| Measure, don't guess | Read `scannedBytes` in the query result before and after a change | Recommended |
| `==` for exact matches, `~` for wildcards only | `field == "exact"` is faster than `field ~ "exact"` | Recommended |
| Combine conditions in single filter | `filter span.kind == "server" and dt.service.name == "checkout" and duration > 100ms` | Recommended |
| Substring tests after selective filters | `contains(url.path, …)` skips little data on its own | Recommended |
| Select only needed fields | Smaller results — scanned bytes are unchanged | Optional |
| Compute derived fields after filtering | `fieldsAdd duration_ms = ...` must appear after all `filter` steps | Recommended |
| Sort after summarize, never after fetch | Sorting before aggregation wastes resources | Critical |
| Limit after sort, at pipeline end | `sort desc \| limit 20` as final commands — never `limit` before `summarize` | Critical |

# 3. Time Range & Cost

| Practice | Recommended Setting/Value | Priority |
| --- | --- | --- |
| Always specify explicit time range | `fetch spans, from:-1h` — never bare `fetch spans` | Critical |
| Use narrowest range possible | Debugging: `from:-5m`. Recent: `from:-1h`. Daily: `from:-24h`. Weekly: `from:-7d` (1 h → 15 min cut 910 MB to 234 MB) | Critical |
| Target specific buckets | `fetch spans, from:-1h, bucket:{"<your bucket>"}` — check `dt.system.bucket` first | Recommended |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Read sampling for estimates | `fetch spans, samplingRatio: 100` then multiply counts by `dt.system.sampling_ratio` (the read-sampling ratio, not the trace sampler) | Optional |
| Never group by `trace.id` without pre-filtering | Millions of unique values — filter by error or time first | Critical |
| Group by low-cardinality fields for dashboards | `dt.service.name`, `span.name`, `span.kind`, `http.route` — never `url.path`, `trace.id` | Recommended |
| Always `limit` non-aggregated queries | `limit 100` on any raw span query | Critical |
| Use only needed percentiles | Each adds compute time, not scanned bytes | Optional |

## 4. Duration & Aggregation

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Convert duration to a number | `duration / 1ms`, `duration / 1s` — never `/ 1000000` (still a duration) | Critical |
| Use duration literals in filters | `filter duration > 100ms` — an integer literal never matches a duration | Critical |
| Use `countIf()` for conditional counts | `summarize errors = countIf(span.status_code == "error"), total = count()` in single pass | Recommended |
| Error rate calculation | `(error_count * 100.0) / total_requests` — use `100.0` (float) to avoid integer division | Recommended |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Percentile trends | `makeTimeseries p95_ms = percentile(duration / 1ms, 95), interval:10m` — conversion **inside** the aggregation; `percentile(duration, 95) / 1ms` is rejected | Critical |
| Use `makeTimeseries` for dashboard charts | `makeTimeseries {requests = count(), errors = countIf(span.status_code == "error")}, interval:5m` | Recommended |
| Server error rate | Count `span.status_code == "error" or http.response.status_code >= 500` — 7.7% of 5xx server spans carried no status on the validation tenant | Recommended |
| Minimum sample size before ranking | `filter request_count > 10` before percentile/error rate rankings | Recommended |

# 5. Trace Analysis

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Find root spans | `filter isNull(span.parent_id)` | Recommended |
| Reconstruct trace chronologically | `filter trace.id == toUid("ID") \| sort start_time asc` | Recommended |
| Find the ORIGIN error in a trace | The error span that **ends first**: `summarize origin = takeMin(record(end_time = end_time, service = dt.service.name, span = span.name)), by:{trace.id}`. The earliest-*starting* error is usually a caller that inherited it (231 of 247 multi-error traces) | Critical |
| Trace duration | `(max(end_time) - min(start_time)) / 1ms` per `trace.id` — not `sum(duration)` (double-counts nested work) | Recommended |

| Practice | Recommended Setting/Value | Priority |
| --- | --- | --- |
| Detect cascading failures | `filter span.status_code == "error" \| summarize n = count(), by:{trace.id} \| filter n > 1` | Recommended |
| Impact score for bottleneck spans | `fieldsAdd impact_score = call_count * avg_duration_ms \| sort impact_score desc` | Recommended |
| `sum(duration)` per service is inclusive time | It includes child calls a span waits on — rank by it, don't call it self time | Recommended |
| Health scorecard classification | `if(error_rate > 5, "Critical", else: if(error_rate > 1, "Warning", else: "Healthy"))` | Recommended |

# 6. Service Dependencies

| Practice | Recommended Setting/Value | Priority |
| --- | --- | --- |
| Map dependencies via CLIENT spans | `filter span.kind == "client" \| summarize n = count(), by:{dt.service.name, server.address}` | Recommended |
| Caller → callee service map | Join server spans to the client span named in `span.parent_id` (same `trace.id`) — see SPANS-04 §3 | Recommended |
| Don't rely on `peer.service` | No semantic-dictionary row; 0 of 8.29M spans in 24 h on the validation tenant | Critical |
| Track async messaging | `filter span.kind == "producer" or span.kind == "consumer" \| summarize n = count(), by:{messaging.system, messaging.destination.name}` (`messaging.operation.type`, not `messaging.operation`) | Recommended |
| Outbound/inbound ratio | `toDouble(outbound) / toDouble(inbound)` per `dt.service.name` — integer division truncates | Optional |

# 7. Security

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use `http.route` in dashboards, never `url.path` | `http.route` = pattern (no PII), `url.path` = actual values (may contain PII) | Critical |
| Audit query strings where they live | Tokens sit in `url.query` / `url.full`, not `url.path` — check all three | Critical |
| Mask card numbers as whole path segments | `replacePattern(url.path, "<<'/' CREDITCARD (>>'/' \| >>'?' \| EOF)", "[CARD-MASKED]")` — a bare `CREDITCARD` also matches digit runs inside hex IDs | Critical |
| Weekly PII audit: emails in URLs | Leak check: `countIf(replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "") != url.path)` — must be 0 | Critical |
| Weekly audit: sensitive data in `db.query.text` | `contains(db.query.text, "password", caseSensitive: false)` and similar (`db.statement` is the null pre-1.0 name) | Critical |
| Exceptions are span events | `iAny(span.events[][span_event.name] == "exception")`; text in `exception.message` / `exception.stack_trace` | Recommended |
| Monitor 401/403/429 as security signals | `filter in(http.response.status_code, {401, 403, 429})` with 10-min bucketing on dashboard | Critical |
| Brute force detection | 401 count >10 per hour per endpoint = alert (community threshold — tune it) | Recommended |
| Enumeration detection | 404 count >100 with high unique path count = scanning alert (community threshold) | Recommended |
| Limit `trace.id` exposure | `takeAny(trace.id)` per error pattern — never expose all trace IDs | Recommended |
| Mask emails in span URLs via OpenPipeline | DQL processor: `replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "[EMAIL-MASKED]")` | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Mask tokens in query params via OpenPipeline | DQL processor: `replacePattern(url.query, "<<(('token' \| 'TOKEN' \| 'Token' \| 'key' \| 'KEY' \| 'Key' \| 'secret' \| 'SECRET' \| 'Secret' \| 'password' \| 'PASSWORD' \| 'Password') '=') [^&]+", "[REDACTED]")` — DPL literals are case-sensitive, so list each casing | Critical |

# 8. OpenPipeline Span Configuration

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Drop health check spans at ingestion | **Drop record** processor, matching condition `matchesValue(span.name, "*health*") or matchesValue(span.name, "*/ready*")` (matchers reject `contains()`) | Critical |
| Drop static asset spans | Drop record on `matchesValue(url.path, "*.js") or matchesValue(url.path, "*.css")` — measure first | Recommended |
| Never drop error or slow spans | Leave `span.status_code == "error"` and `duration > 1s` out of every drop condition | Critical |
| Pre-compute `duration_ms` at ingestion | DQL processor: `fieldsAdd duration_ms = duration / 1ms` | Optional |
| Route to environment buckets | Bucket assignment (first match): production → long-retention bucket, staging → short, `true` last | Recommended |
| Route sensitive service spans to restricted buckets | Payment/auth services → `spans_sensitive` with restricted IAM | Recommended |
| Bucket-level IAM for team isolation | Frontend team: `spans_frontend` only. SRE: all span buckets. Compliance: `audit_spans` | Recommended |

# 9. Bucket Strategy & Sampling

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Retention by environment | Production: 90 days. Staging: 7 days. `default_spans` is built in (10 days on the validation tenant); Grail traces are *"Configurable, from 10 days to 10 years"* | Recommended |
| Sample at the source, not in OpenPipeline | OneAgent: Adaptive Traffic Management (head-based). OpenTelemetry: Collector `tail_sampling` — the only place "keep all errors" is possible. *"OpenPipeline processing cannot be used as an alternative to ATM configuration for controlling trace volume."* | Critical |
| Compute metrics before tail sampling | `spanmetrics` connector ahead of `tail_sampling` in the Collector | Recommended |
| Calculate savings before filtering | Count health checks + static assets ( `url.path` ) vs errors + slow to size droppable vs must-keep | Recommended |
| Extract `dt.ingest.size` as metric | **Sampling aware value metric** (the value-metric variant on spans), key `span.ingest.size.by.app`, dim: app identifier | Recommended |
| Verify bucket distribution monthly | `summarize n = count(), by: {dt.system.bucket}` — verify routing rules working | Recommended |

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*